Lehrstuhl für Software und Systems Engineering
TUM School of Computation, Information and Technology
Technical University of Munich

# Agentic AI for Code Generation

Seminar Report

**Student: "Thua Duc Nguyen"** ✉ **and Supervisor: "Derui Zhu"** ✉

TUM School of Computation, Information and Technology, Technical University of Munich

✉ thuaduc.nguyen@tum.de
✉ derui.zhu@tum.de

April 1, 2025

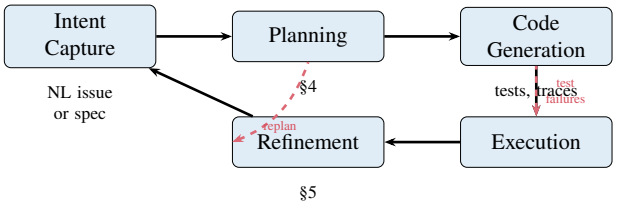


**Figure 1** The vibe coding pipeline. Intent flows through planning, generation, and execution. Test failures trigger refinement; replanning occurs when the initial hypothesis is invalidated.

**Abstract** —

## 1 Introduction

**Motivation.** The rise of "vibe coding" and AI-assisted software development: how natural-language-driven programming is reshaping the developer workflow.

**Background and Related Surveys.** Positioning relative to prior surveys on LLM-based code generation, automated program repair, and AI agent architectures. What this report adds: a pipeline-centric synthesis across four active research fronts.

**Scope and Contribution.** Define the scope (autonomous code-generation agents, 2023–2025 literature), state the contribution (unified pipeline framing, cross-topic synthesis), and outline the report structure.

## 2 The Vibe Coding Pipeline

Present the vibe coding pipeline as a unifying conceptual framework and compare it to classical SE process models (waterfall, agile inner loops).

### 2.1 Pipeline Stages

The vibe coding pipeline (Figure 1) consists of five stages forming a loop: Intent Capture → Planning → Code Generation → Execution → Refinement. Execution feedback drives refinement, which may trigger re-planning or re-generation. Adjacent stages exchange structured artefacts: natural-language intent and repository context flow into plans and candidate patches; generated code is run against test oracles; failures and traces feed back into replanning or repair.

### 2.2 Mapping to Survey Topics

Explicit mapping: how each of the three subsequent sections corresponds to one or more pipeline stages. §3 covers the full pipeline as an integrated system—including action interfaces for code generation (ACI, code-as-action, diff-based edits); §4 zooms into planning, candidate search, and the plan–generate interface; §5 covers execution environments, test-driven feedback, and the refinement loop (including fault-specific self-repair). Code generation is distributed across §3 and §4 rather than isolated in a dedicated section. Cross-cutting concerns (evaluation, context management) are consolidated in the Discussion. Figure 2 provides a document-wide taxonomy: report sections mapped to pipeline stages, with representative systems and techniques as leaf exemplars.

## 3 End-to-End Autonomous Software Engineering Agents

Before examining individual pipeline stages, we survey systems that orchestrate the *entire* vibe coding pipeline, accepting a natural-language issue or specification and producing a tested code change with minimal human intervention. These end-to-end agents provide the holistic context in which planning (§4) and feedback-driven refinement (§5) operate.

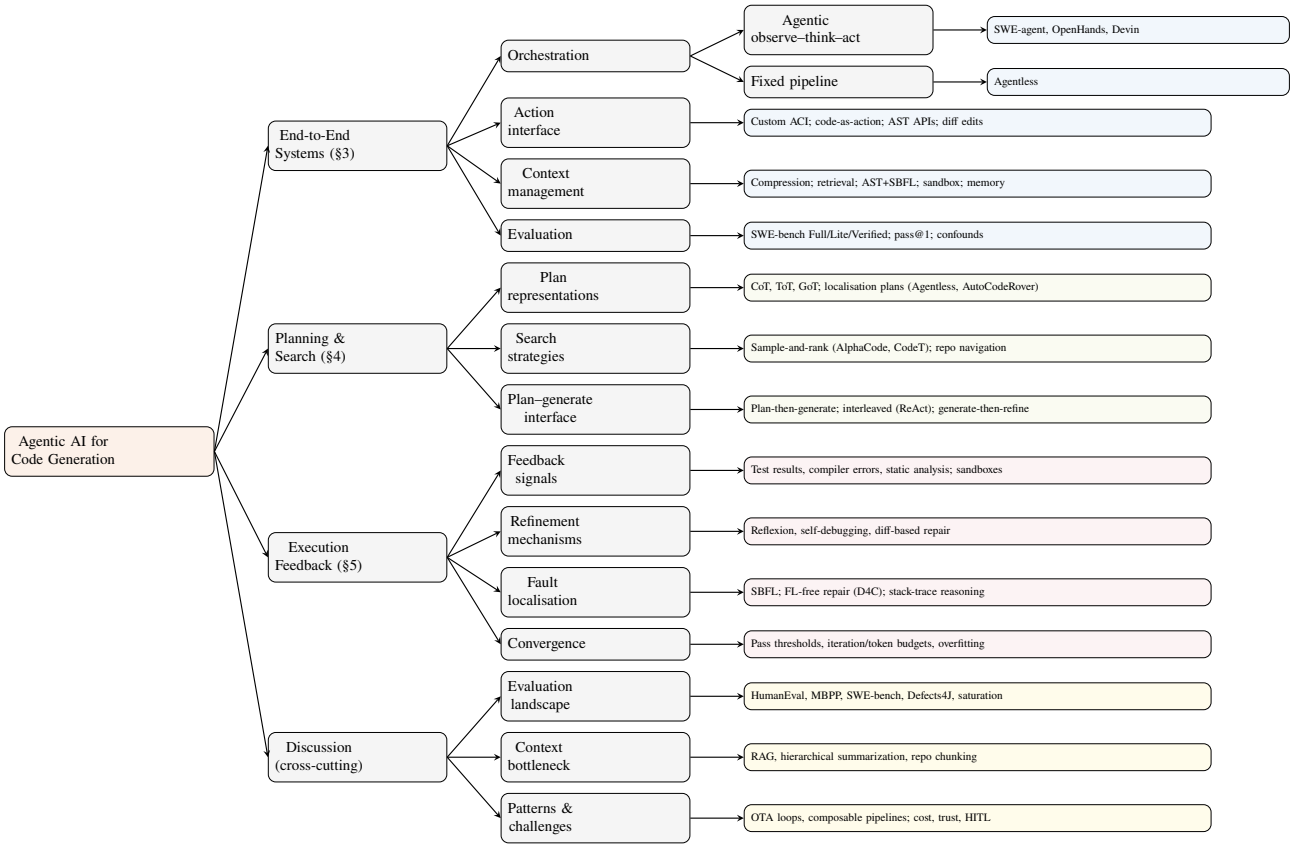


**Figure 2** Survey taxonomy organised by report section and vibe coding pipeline stage. §3 surveys integrated end-to-end systems; §4 and §5 zoom into planning/search and execution-feedback mechanisms; the Discussion consolidates evaluation, context limits, and open challenges. Leaf nodes name representative systems or techniques.

We begin by identifying the architectural patterns that underlie most current systems (§3.1). We then survey five representative agents that span the design space: from fully autonomous to deliberately non-agentic (§3.2). Finally, we examine how these systems are evaluated, focusing on SWE-bench and its variants (§3.3). This section instantiates the *End-to-End Systems* branch of Figure 2.

## 3.1 Architecture and Orchestration

Despite surface-level diversity, autonomous SE agents converge on a small number of architectural patterns. As illustrated in the *Orchestration*, *Action Interface*, and *Context Management* branches of Figure 2, these patterns determine how agents perceive their environment, what actions they can take, and how they manage repository-scale context.

**The observe–think–act loop.** The dominant paradigm is a ReAct-style [25] loop in which the agent alternates between (i) observing the environment (file contents, test output, error traces), (ii) reasoning about what to do next (chain-of-thought or structured plan), and (iii) executing an action via a tool call. SWE-agent [23] and OpenHands [18] both instantiate this loop, differing mainly in the *action space* exposed to the model (Figure 3).

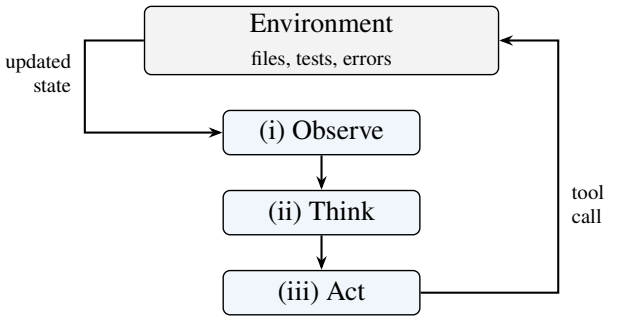


**Figure 3** ReAct-style observe–think–act loop. The agent reads environment state, reasons about the next step, executes a tool call, and repeats.

**Agent–Computer Interface (ACI).** Yang et al. [23] introduce the ACI concept, arguing that LLMs are a "new category of end users" that need purpose-built interfaces rather than raw shell access. SWE-agent's ACI includes a windowed file viewer (100 lines), a linter-

**Table 1** Comparison of end-to-end autonomous SE agents. Architectural patterns: **OTA** = observe–think–act loop, **ACI** = custom Agent–Computer Interface, **CaA** = code-as-action, **Pipe** = fixed multi-phase pipeline (no agent loop). SWE-bench scores are single-pass (pass@1) unless noted; **Lite** = SWE-bench Lite (300 instances), **Full** = SWE-bench Full (2,294 instances). Cost is average USD per task instance.

| System | Origin | Architecture | Context Strategy | Lite | Full | Cost |
|---|---|---|---|---|---|---|
| SWE-agent | Princeton | OTA + ACI | Observation compression | 18.0% | 12.5% | $1.21 |
| Agentless | UIUC | Pipe | Hierarchical skeleton + embeddings | 32.0% | – | $0.70 |
| AutoCodeRover | NUS | OTA + AST search | AST-based APIs + SBFL | 19%[†] | 12.4% | $0.43 |
| OpenHands | Multi-inst. | OTA + CaA | Docker sandbox + delegation | 26.0% | – | – |
| Devin | Cognition | OTA + Brain/DevBox | Persistent knowledge store | –[‡] | – | propr. |

[†]26% with 3 retries (pass@3). [‡]Proprietary; see text for Verified estimates.

guarded `edit` command, summarised search results, and compressed observation history. Ablations show that each ACI component contributes measurably: removing the linter drops performance by 3 percentage points; switching to iterative search costs 6 pp; and showing full file content instead of a 100-line window costs 5.3 pp on SWE-bench Lite.

**Code-as-action.** OpenHands adopts the CodeAct paradigm [17], consolidating all agent actions into executable Python and bash code rather than a fixed tool API. Each turn may run arbitrary IPython cells, bash commands, or browser interactions inside a Docker-sandboxed environment. This design maximises flexibility at the cost of a larger action space that the model must navigate.

**Context management.** Repository-level tasks easily exceed LLM context windows. Agents employ several mitigation strategies: SWE-agent collapses older observations beyond the last 5 turns; AutoCodeRover [26] parses the codebase into an AST and exposes structure-aware search APIs (`search_class`, `search_method_in_class`) that return only signatures, not full definitions; Agentless [21] constructs a hierarchical repository skeleton and uses embedding-based retrieval to select candidate files before prompting the LLM.

## 3.2 Representative Systems

We survey five prominent systems that span the design space. Table 1 provides a comparative overview.

**SWE-agent [23].** Developed at Princeton, SWE-agent wraps a GPT-4 Turbo backbone in a custom ACI with specialised file navigation, editing, and search commands. It achieved 12.47% on the full SWE-bench test set (2,294 tasks) and 18.00% on SWE-bench Lite when first published. The agent follows a typical trajectory: reproduce the issue, localise the faulty code via `find_file`/`search_dir`, then enter an edit–execute loop. The median successful run completes in 12 steps at $1.21 API cost.

**Agentless [21].** From UIUC, Agentless challenges the need for agentic autonomy. It replaces the interactive loop with a fixed three-phase pipeline: *localise* → *repair* → *validate*. Localisation uses a hierarchical narrowing strategy (files → classes/functions → edit locations) combining LLM prompting with embedding retrieval. Repair generates up to 40 candidate patches in a search/replace diff format. Validation selects the best patch via regression testing, LLM-generated reproduction tests, and majority voting over AST-normalised candidates. Agentless resolved 32.00% of SWE-bench Lite at an average cost of only $0.70 per problem, making it the top open-source system at the time and establishing a strong non-agentic baseline. OpenAI later adopted Agentless as its default scaffold for showcasing GPT-4o and o1 on SWE-bench Verified.

**AutoCodeRover [26].** From NUS, AutoCodeRover takes a "software-engineering-oriented" approach, operating on program representations rather than raw files. It parses the codebase into an AST and provides the LLM with structure-aware search APIs. A stratified context retrieval process iteratively expands the agent's understanding, starting from issue keywords and refining through multiple search rounds until sufficient context is gathered. When a test suite is available, Spectrum-Based Fault Localisation (SBFL) augments the search with suspiciousness scores. AutoCodeRover achieved 19% on SWE-bench Lite in a

single run and 26% with three retries, at a cost of $0.43–$1.30 per task.

**OpenHands [18].** Formerly OpenDevin, OpenHands is an open-source platform (ICLR 2025, 32K GitHub stars, 188 contributors) built around the CodeAct agent. Each session runs inside a Docker container with a bash shell, IPython server, and Chromium browser. The platform supports multi-agent delegation—a generalist CodeAct agent can offload web browsing to a specialised BrowsingAgent. On SWE-bench Lite, CodeAct v1.8 with Claude 3.5 Sonnet achieved 26.0%. On SWE-bench Verified, a later submission reaches 60.6% with a single rollout and 66.4% when five trajectories are reranked by a dedicated critic model [14].

**Devin (Cognition) [5].** Devin is a proprietary cloud-hosted agent positioned as an "autonomous AI software engineer." Its architecture separates a *Brain* (reasoning layer) from a *DevBox* (execution VM with shell, IDE, and browser). A persistent Knowledge store provides cross-session memory. Cognition reports strong full-benchmark results, but independent leaderboard analysis finds 68.3% on SWE-bench versus 48.2% on SWE-bench Verified (a 29% relative drop), suggesting possible overfitting to easier unfiltered instances [6].

Taken together, these five systems expose a recurring trade-off between agentic flexibility and pipeline efficiency. Agentless shows that a fixed localise–repair–validate pipeline can outperform interactive agents on SWE-bench Lite at lower cost ($0.70 vs. $1.21), challenging the assumption that autonomy is necessary for strong results. Among agentic designs, interface and context strategy matter as much as the backbone LLM: SWE-agent's ACI ablations, OpenHands's CodeAct action space, and AutoCodeRover's AST search APIs each demonstrate that how an agent reads and edits code can shift resolve rates by several percentage points at comparable model capacity.

### 3.3 Full-Pipeline Evaluation

The *Evaluation* branch of Figure 2 summarizes the primary benchmark splits, scoring protocols, and known confounds that shape current performance reports.

**SWE-bench: dataset construction.** The primary evaluation instrument for end-to-end agents is SWE-bench [10]. It was constructed by scraping merged pull requests from 12 popular open-source Python repositories (including Django, scikit-learn, sympy, matplotlib, and Flask). Each PR was filtered to retain only those that modify the project's test suite. For each retained instance the benchmark provides: (i) the repository snapshot *before* the PR was merged, (ii) the issue description (title + body), (iii) a gold patch (the human-written fix), and (iv) a set of *fail-to-pass* tests (which fail before the fix and should pass after) and *pass-to-pass* tests (which must continue to pass). An agent receives only items (i) and (ii); its generated patch is applied and scored against both test sets.

**Table 2** SWE-bench dataset splits.

| Split | Size | Selection criterion |
|---|---|---|
| Full | 2,294 | All filtered PRs |
| Lite | 300 | Subset selected for tractability |
| Verified | 500 | Human-validated |

**Splits and filtering**: **Full** comprises all filtered instances. **Lite** selects 300 instances for faster iteration, but Xia et al. [21] found quality issues—including gold-patch leakage in issue text and tasks that lack sufficient information to be solvable. **Verified** [13] addresses these concerns: human annotators assessed each Full instance for problem clarity, test correctness, and solvability, retaining 500 high-quality instances that have become the de facto leaderboard standard [7].

**Task complexity profile.** Although SWE-bench tests real-world issues, most tasks are relatively small [8, 7]. In the Verified split, human annotators estimated that 39% of tasks are trivial (<15 min fix), 52% are small (15 min–1 h), 8% are substantial (1–4 h), and only 3 tasks were rated >4 h. These estimates correlate with patch size: easy tasks average 5 changed lines of code (LOC) and 1.0 files; medium tasks average 14 LOC and 1.3 files; hard tasks average 56 LOC and 2.0 files. Crucially, 86% of instances are single-file changes, and 87% are classified as bug fixes rather than feature requests or refactorings. The dominance of Django (nearly 50% of instances) and four other repositories (>80% combined) further limits diversity.

**Leaderboard trajectory.** Figure 4 charts the rapid progress on SWE-bench from late 2023 through mid-2026. In October 2023, the best RAG baselines resolved 1.96% of Full and 0.33% of Lite [10]. SWE-agent's agentic loop lifted Full to 12.5% and Lite to 18.0% in early 2024. On the Full set, AutoCodeRover

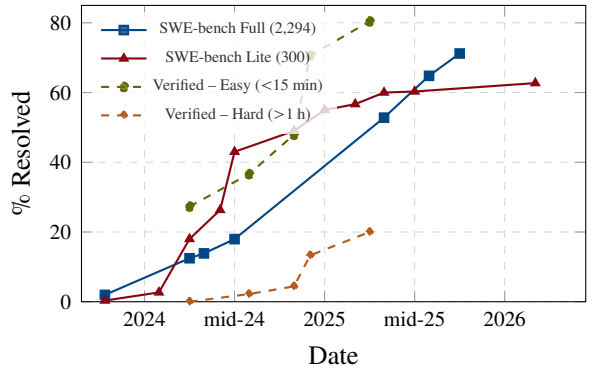


**Figure 4** SWE-bench progress over time. Solid lines: best-in-class resolve rate on the Full set (blue, 2,294 tasks) and Lite (red, 300 tasks). Dashed lines: performance on easy (<15 min, green) vs. hard (>1 h, orange) subsets of Verified, illustrating the widening difficulty gap [8]. Data compiled from leaderboard submissions [16, 6, 1].

reached 18.0% (pass@3) and Devin 13.9% around the same period. The scaffolding race then accelerated both tracks: by late 2024, Agentless 1.5 with Claude 3.5 Sonnet reached 40.7% on Lite; by mid-2025 the best Lite score stood at 60.3%. On the Full 2,294-instance set, fewer teams report results post-2024, but dataku's independent analysis [6] records Claude Opus 4 at 71.2% and Devin at 68.3%. On Lite, the frontier reached ~63% by early 2026 [16].

The easy–hard gap is striking: the best individual system solves ~80% of easy tasks (<15 min) but only ~20% of hard ones (>1 h) [8]. Combined across all top systems, 95% of easy instances have been resolved by at least one agent, versus only 42% of hard ones—and among hard multi-file issues, only 9 of 25 have ever been solved. This ceiling effect, combined with contamination risk (half the issues predate 2020 and the repositories are widely used as training data [7]), raises concerns about the benchmark's continued discriminative power. SWE-bench Pro, a harder leaderboard with novel uncontaminated problems, shows Claude Opus 4.5 dropping to ~46% [1].

**Beyond SWE-bench.** SWE-bench covers only Python bug-fixing in 12 repositories. Newer benchmarks begin to address broader scopes: SWE-bench Multilingual (300 tasks, 9 languages), CodeAgentBench (interactive development environments), and real-world deployment metrics. A comprehensive discussion of the evaluation landscape is deferred to §6.2.

# 4 Planning and Search Strategies for Code Synthesis

Planning is the stage in which an agent turns a vague intent into an operational hypothesis about *what* code should change and *how* the change can be produced. In the vibe coding pipeline, this stage sits between intent capture and code generation, but in practice it is rarely a single up-front step. Agents plan while reading issue reports, searching repositories, choosing files, decomposing edits, sampling candidate patches, and revising their strategy after failed tests. Thus, planning is best understood as a control layer over generation rather than as a separate document written before generation begins. While §3 evaluates end-to-end agents primarily on SWE-bench, the planning and search mechanisms below are studied across function-level benchmarks (HumanEval, MBPP), competitive programming (CodeContests), and repository repair tasks alike. This section instantiates the *Planning & Search* branch of Figure 2.

This section surveys three aspects of that control layer:

1. Plan representations: determine what intermediate state the agent exposes to itself: natural-language rationales, hierarchical task lists, program sketches, or repository-localisation hypotheses;
2. Search strategies: determine how many possible solutions are explored and how candidates are ranked;
3. The plan-generate interface: determines whether planning happens once, repeatedly, or only when execution feedback invalidates the current hypothesis.

## 4.1 Plan Representations and Decomposition

Plan representations determine how an agent organises reasoning before generating code. The simplest is chain of thought [20]: a linear sequence that commits early to one trajectory. If the first design choice is wrong, subsequent steps elaborate the mistake. Tree of Thoughts [24] and Graph of Thoughts [2] relax this by allowing exploration, backtracking, and merging across alternative paths (Figure 5). For code synthesis, this means agents can consider multiple decomposition strategies—"change the parser", "change the caller", or "adjust the compatibility layer"—as explicit search states rather than implicit in a single sample.

**Table 3** Planning and search mechanisms in LLM-based code synthesis.

| Mechanism | Planning object | Role in code tasks |
|---|---|---|
| Chain / tree / graph of thoughts | Intermediate reasoning steps | Explore alternative designs, algorithms, or debugging hypotheses before emitting code |
| Hierarchical decomposition | Subtasks, files, functions, edit locations | Reduce repository-scale tasks into bounded local edits |
| Candidate sampling | Multiple complete or partial programs | Trade inference cost for higher pass@k and diversity |
| Execution-guided selection | Tests, generated tests, compiler output | Rank or prune candidates using external evidence rather than model confidence alone |
| ReAct-style replanning | Updated belief state after tool observations | Interleave search, reading, editing, and feedback in a single loop |

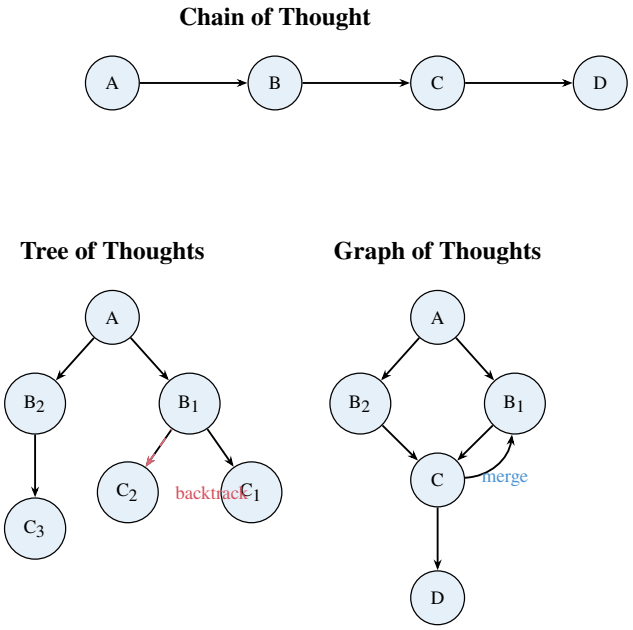


**Figure 5** Plan representation structures. Chain of Thought commits to a single linear path. Tree of Thoughts explores alternatives and backtracks. Graph of Thoughts merges branches and revisits earlier states.

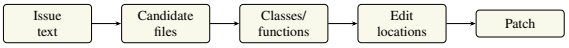


**Figure 6** Hierarchical localisation pipeline. Agents narrow scope iteratively from repository-level to line-level edits.

Repository-level agents usually add a more concrete representation (recall §3.2): *localisation plans*. Agentless decomposes issue resolution into files, classes/functions, and edit locations before generating a patch [21]. AutoCodeRover uses AST-backed APIs such as `search_class` and `search_method_in_class` to plan over program structure rather than raw text [26]. These grounded representations name concrete program elements that can be validated against the repository (Figure 6), turning a broad intent into a bounded search problem.

Plans may also be expressed as *specification sketches*: partial interfaces, pseudocode, assertions, or examples that constrain the generated implementation. Useful plans are grounded in observable artefacts—issue text, existing code, API conventions, and failing tests—rather than invented requirements.

## 4.2 Search and Exploration Strategies

Once a representation exists, an agent must decide how broadly to search. Single-sample generation is fast, but it treats the model's first completion as the solution. Search-based systems instead spend additional inference or execution budget to explore alternatives. The simplest form is self-consistency: sample multiple reasoning paths and select the most common or most consistent answer [19]. For programming, the same idea becomes sample-and-rank: generate several candidate programs or patches, then choose using tests, static checks, model judges, or majority voting. AlphaCode [11] and CodeT [3] exemplify this pattern (Figure 7). AlphaCode generated millions of candidates for competitive programming, filtered and clustered by execution results, then submitted a small diverse set. CodeT generates both programs and tests, selecting code consistent with generated test cases. The key insight is that LLMs often contain the right solution somewhere in the sample distribution even when top-ranked output is wrong. The limitation is self-referential objectives: if generated tests are weak, selection optimises for the model's own misunderstanding. Execution-guided search is strongest when feedback includes external oracles—existing unit tests, compiler errors, type checkers, or repository-specific regression suites.

Search also appears in repository navigation, as in the systems surveyed in §3. Agentless samples multiple locations and candidate patches, then normalises patches by AST and uses validation to pick among them [21]. AutoCodeRover expands context iteratively through program-structure queries, optionally using Spectrum-Based Fault Localisation (SBFL) to prioritise suspicious code when tests are available [26]. In both cases, search is not over complete programs alone; it is also over *where to look* and *what evidence to include*. This matters because code generation quality is bounded by context quality. A correct patch is

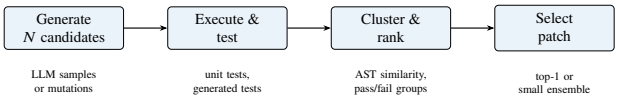


**Figure 7** Candidate-selection pipeline. Systems like AlphaCode and CodeT generate many candidates, execute them against tests (real or generated), cluster by behaviour, and select a small high-quality set.

**Table 4** Plan–generate coupling patterns.

| Pattern | Mechanism | Trade-off |
|---|---|---|
| Plan-then-generate | Plan once, then generate code conditioned on it | Easy to inspect; brittle if plan is wrong |
| Interleaved planning | Alternate plan updates and tool use in observe–think–act loop | Adaptive; plan distributed across turns |
| Generate-then-refine | Generate, critique via feedback, revise iteratively | Self-correcting; may reopen planning after execution |

unlikely if the agent never places the relevant function, invariant, or failing trace in context.

The cost of search is the main trade-off. More samples increase diversity and pass@k, but they also increase latency, token cost, and validation cost. Moreover, search can overfit to narrow tests: a candidate that passes the available suite may still violate unstated requirements. Strong systems therefore combine multiple pruning signals: syntactic validity, static analysis, failing-test repair, pass-to-pass regression preservation, and semantic similarity to the intended behaviour. Search is most useful when the ranking signal is at least as reliable as the generator is diverse.

### 4.3 Plan–Generate Interplay

Planning and generation can be coupled in three broad ways (Table 4). *Plan-then-generate* is easy to inspect and aligns with human practice, but brittle if the initial plan is wrong [21]. *Interleaved planning* adapts to new information but distributes the plan across many turns, making it harder to audit [25]. *Generate-then-refine* treats the first output as a hypothesis and revises via self-critique or execution feedback [12], connecting planning to the refinement loop in §5.

The central design question is not whether agents should plan, but how plans remain *grounded*. Grounding mechanisms include retrieval over repository structure, AST-level symbol search, execution traces, failing tests, and explicit constraints extracted from the issue report. Effective systems keep plans close to these artefacts and treat generation as one move in a larger search process, reframing code synthesis from "write the answer" to "maintain and test a hypothesis about the required program change".

## 5 Execution Feedback and Iterative Refinement

Building on the plan–generate patterns surveyed in §4, this section examines how execution closes the loop. Code generation rarely succeeds in one shot. Even human programmers iterate: write, compile, test, debug, revise. Agentic systems execute generated code, observe runtime outcomes—test failures, compiler errors, exceptions—and revise their output accordingly. While §3 scores agents on SWE-bench, refinement mechanisms are often evaluated on function-level suites (HumanEval, MBPP), text-to-SQL (Spider), and APR benchmarks (Defects4J). This section instantiates the *Execution Feedback* branch of Figure 2 and examines the feedback signals agents consume (§5.1), the mechanisms by which they refine code (§5.2), the techniques for localising and repairing faults (§5.3), and the criteria for deciding when to stop iterating (§5.4).

### 5.1 Feedback Signals and Environments

Agents require feedback to improve. The type, source, and presentation of feedback critically affect refinement efficacy.

**Types of feedback.** The most common feedback is *test execution results*: pass/fail verdicts, failing test cases, assertion messages, and expected-versus-actual output diffs. When tests fail, agents receive concrete evidence of incorrectness. A richer signal is *compiler or interpreter errors*: syntax errors, type mismatches, undefined-variable messages, and stack traces. These pinpoint the location and nature of faults without requiring test design. Static analysis tools provide another layer: linters flag style violations, type checkers identify inconsistencies, and data-flow analysers detect null dereferences or race conditions before execution. Some systems incorporate *human feedback* in the loop: developers approve or reject patches, provide natural-language explanations of failures, or annotate bug reports. Finally, agents can generate *self-critique*:

an LLM explains the generated code in natural language, identifies logical flaws, or predicts potential failures without executing the code ("rubber duck debugging") [4].

**Feedback sources: external versus internal.** Feedback may come from *external* oracles—prewritten test suites, continuous-integration pipelines, runtime environments—or be *internally simulated* by the agent itself. External feedback is authoritative but requires infrastructure and may be slow or expensive. Internal feedback is fast and requires no setup, but risks hallucination: the agent may incorrectly believe its code is correct or fabricate error messages. Hybrid approaches use external execution for validation and internal self-critique for diagnosis.

**Sandboxed execution environments.** To safely run untrusted generated code, systems employ sandboxing. Common techniques include *containerisation* (Docker), *virtual machines*, *language-level isolation* (restricted Python interpreters), and *timeout and resource limits*. SWE-agent and AutoCodeRover run generated code in Docker containers with filesystem restrictions and network isolation. Reflexion [15] uses a Python interpreter with timeout guards and exception handling. Sandboxing prevents malicious code from damaging the host system, but introduces latency and limits access to system resources, databases, or network services that the real code may need. Balancing safety and realism is an ongoing design challenge.

## 5.2 Iterative Refinement Mechanisms

Once feedback is available, agents must incorporate it to produce better code. Refinement strategies vary in how they represent feedback, how they prompt the model to revise, and whether they maintain memory across iterations.

**Reflexion: verbal reinforcement learning.** Reflexion [15], from Northeastern, MIT, and Princeton, treats refinement as a form of reinforcement learning without weight updates. After each trial, the agent reflects on what went wrong by generating a natural-language critique ("The function failed because it did not handle negative inputs"). This reflection is stored in an episodic memory buffer and retrieved in subsequent attempts, conditioning the model to avoid past mistakes. Reflexion accepts diverse feedback: scalar rewards (e.g., pass rate), binary verdicts, or free-form error messages. The key innovation is converting feedback into *verbal* form that persists across trials, enabling the model to learn from experience within a single episode. On HumanEval, Reflexion achieves 91% pass@1, surpassing single-shot GPT-4 (80%) by explicitly encoding trial-and-error history. The approach generalises beyond code: it improves sequential decision-making in games and language reasoning tasks.

**Self-debugging: rubber duck debugging.** Chen et al. [4] propose *self-debugging*, where the model critiques and repairs its own code without external feedback. The agent generates code, then *explains* the code line-by-line in natural language. This explanation serves as a self-generated specification, allowing the model to spot discrepancies between intent and implementation. If the explanation reveals a flaw ("This loop iterates over indices, but I wanted to iterate over values"), the model revises the code. Self-debugging also incorporates external execution feedback when available: the model receives error messages or failing test outputs and reasons about root causes before proposing fixes. On Spider (text-to-SQL), self-debugging improves baseline accuracy by 2–3% without unit tests and by up to 12% on MBPP when tests are available. The method demonstrates that internal self-critique complements external execution signals.

**Repair prompts and diff-based editing.** Simpler refinement strategies use *repair prompts*: the agent receives the original task, the buggy code, and the error message, then generates a revised version. To reduce token costs, some systems use *diff-based editing*, where the model outputs only the changes (insertions, deletions) rather than the full code. This is particularly effective for large files: instead of regenerating hundreds of lines, the agent specifies "replace line 42" or "insert after line 57". SWE-agent's edit tool and Agentless's diff-based repair both adopt this paradigm. The trade-off is that models must learn the diff format, and parsing errors can break the repair loop.

**Multi-turn conversation with the environment.** Some agents treat refinement as a dialogue with the execution environment. The agent generates code, the environment returns feedback, the agent asks clarifying questions or requests additional context ("What is the expected output for negative inputs?"), the environment answers, and the cycle repeats. This conversa-

tional pattern mirrors human debugging sessions with a collaborator. However, it assumes the environment can answer semantic questions, which is typically not the case for automated test suites. In practice, multi-turn systems interact with a supervisor LLM that interprets test failures and generates explanations, adding an extra layer of inference cost.

### 5.3 Fault Localization and Self-Repair

When refinement targets bugs in existing code rather than improving draft solutions, it becomes *automated program repair* (APR). The classic APR pipeline has three stages: fault localisation (identify buggy lines), patch generation (propose fixes), and patch validation (test the fix). LLM-based agents challenge this structure by generating end-to-end repairs.

**Traditional fault localisation.** Pre-LLM APR systems rely on *spectrum-based fault localisation* (SBFL), which ranks code lines by suspiciousness scores derived from test coverage: lines executed by failing tests and not by passing tests receive high scores. Ochiai, Tarantula, and DStar are standard SBFL metrics. LLM-based systems initially adopted this pattern, using SBFL to extract a small code snippet and prompting the model to repair it. However, fault localisation is imperfect: even oracle-provided bug locations ("perfect FL") do not guarantee correct repairs, and SBFL scores are often noisy.

**FL-free repair with objective alignment.** Recent work questions whether explicit fault localisation is necessary. D4C (Direct Debug Drives Decent Code) [22] argues that LLMs should repair entire functions rather than isolated hunks, aligning the task with their pre-training objective (next-token or infilling completion). D4C prompts the model with the full function, failing test cases, and error messages, and asks it to output a corrected function. On Defects4J, D4C repairs 180 single-function bugs with only 10 samples per bug, outperforming systems with perfect fault localisation by 10% and reducing sample count by 90%. The insight is that LLMs, trained on vast corpora of correct code, implicitly learn to locate faults when given sufficient context; forcing them to target specific lines may constrain their reasoning.

**Stack-trace analysis and root-cause reasoning.** When execution fails, agents receive stack traces showing the call chain and exception type. Stack traces are rich signals: they identify the failing function, reveal the exception (NullPointerException, IndexError), and often include variable values. Agents can parse stack traces to infer root causes: "IndexError at line 15 suggests the list is shorter than expected; check loop bounds." SelfEvolve [9] and SWE-agent [23] iteratively refine code by feeding error messages back to the LLM, which reasons about the fault and proposes targeted edits. The challenge is that stack traces can be noisy (multiple exceptions, deep call stacks) and may not reveal semantic bugs (e.g., off-by-one errors that do not crash).

**Test-driven repair loops.** A refinement loop driven by test feedback proceeds as follows: (1) generate initial code, (2) run tests, (3) if any fail, extract failure messages, (4) prompt the model to fix the code, (5) repeat until all tests pass or a budget is exhausted. The stopping condition is critical: systems may iterate 1, 3, 5, or 10 times. Empirically, most gains occur in the first 2–3 iterations; further iterations yield diminishing returns and risk introducing new bugs. SWE-agent's edit–execute loop and Agentless's repair–validate pipeline both iterate until tests pass or a budget is exhausted. The iteration budget balances quality and cost.

**Candidate ranking and ensemble repair.** When multiple repair attempts produce different patches, agents must select the best one. Ranking strategies include: (i) test coverage (choose the patch that passes the most tests), (ii) LLM-as-judge (ask the model to explain and rank patches by confidence), (iii) ensemble voting (generate $N$ patches and pick the most common or most syntactically similar to the original), and (iv) semantic similarity (prefer patches that minimally alter the original code). AlphaRepair and CodeT use candidate ranking with test execution to filter implausible patches before final selection.

### 5.4 Convergence and Stopping Criteria

Iterative refinement must terminate. Agents need stopping criteria that balance solution quality, cost, and the risk of overfitting to weak test suites.

**Pass-rate thresholds.** The simplest criterion is *full test pass*: stop when all provided tests pass. This works well for competitive programming (HumanEval, CodeContests) where test suites are comprehensive. However, in software engineering tasks

(SWE-bench), tests may be incomplete or incorrect. A patch that passes all tests may still be wrong (a "plausible but incorrect" patch), and a correct patch may fail tests due to environment issues. Some systems use a *relaxed threshold*: stop when $\geq 80\%$ of tests pass, trading coverage for faster convergence.

**Diminishing improvement detection.** If successive iterations yield no improvement—measured by test pass rate, execution success, or LLM confidence scores—the agent should stop. For example, if iterations 3, 4, and 5 all produce code that fails the same test with the same error, further iteration is unlikely to help. SelfEvolve monitors improvement deltas: if the pass rate increases by less than 1% across two iterations, refinement halts. This heuristic prevents endless loops where the model oscillates between two incorrect states.

**Budget limits: iterations and tokens.** Practical systems impose hard limits. Iteration budgets (3, 5, 10 rounds) cap the number of refinement cycles. Token budgets limit total inference cost: after consuming a threshold (e.g., 50k tokens), the agent returns its best attempt. Reflexion and Agentless both use iteration caps, while D4C's low sample count (10 tries per bug) implicitly limits token use. The optimal budget depends on task difficulty: simple bugs may resolve in one iteration, while complex logical errors may require many.

**Risk of overfitting to weak test suites.** A critical failure mode is overfitting. If the test suite is weak (few tests, shallow coverage), the agent may generate code that narrowly passes those tests but fails on unseen inputs. This is the "plausible patch" problem in APR: patches that satisfy the validation oracle but do not generalise. Mitigations include: (i) *test augmentation*, where the agent or a separate test generator produces additional test cases to catch edge cases; (ii) *static checks*, requiring generated code to satisfy linting, type checking, or invariant assertions beyond test pass; (iii) *semantic similarity constraints*, penalising patches that radically alter program structure; and (iv) *LLM-based verification*, asking the model to critique the patch or predict whether it will generalise. However, no technique fully eliminates overfitting, and the problem remains open.

**Human-in-the-loop approval.** In high-stakes domains (medical software, financial systems), automated iteration may be unacceptable without human oversight. Systems like OpenHands and Devin allow developers to inspect intermediate outputs, approve or reject patches, and provide guidance before the next iteration. This trades speed for safety and introduces latency, but ensures that convergence criteria align with human judgment rather than proxy metrics.

# 6 Discussion

## 6.1 Consolidated Patterns and Trends

Recurring design patterns across all four topics (observe-think-act loops, tool orchestration, LLM-as-judge verification). Macro trends: from single-shot prompting to agentic scaffolds, from monolithic agents to composable pipelines, from flat CoT to tree-structured search.

## 6.2 The Evaluation Landscape

Cross-cutting view of benchmarks (HumanEval, MBPP, SWE-bench, Defects4J, CrossCodeEval), their coverage of pipeline stages, saturation concerns, and gaps in realism.

## 6.3 The Context Window Bottleneck

How finite context constrains every pipeline stage; retrieval-augmented generation, hierarchical summarization, and repository-level chunking as mitigations.

## 6.4 Open Challenges

Unsolved problems: long-horizon reasoning, specification ambiguity, cost efficiency, trustworthiness, and human-in-the-loop control design.

## 6.5 Implications for Software Engineering Practice

How these advances may reshape developer workflows, team structures, and the economics of software production.

# 7 Conclusion

Summarize key findings across the four research fronts, restate the pipeline framing as a unifying lens, and point to the most promising future directions.

## References


[1] AgentMarketCap. SWE-bench verified: How AI coding agents went from 1.96% to 80.9% in 30 months. https://agentmarketcap.ai/blog/2026/04/09/swe-bench-verified-progress-timeline-2023-2026, 2026. Accessed: 2025-06-02.

[2] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michal Podstawski, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, 2024.

[3] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*, 2022.

[4] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations*, 2024.

[5] Cognition AI. Introducing devin, the first AI software engineer. https://cognition.ai/blog/introducing-devin, 2024.

[6] dataku. The SWE-bench verified leaderboard: Who's actually solving real bugs? https://dataku.ai/blog/swe-bench-verified-leaderboard-who-solving-real-bugs, 2025. Accessed: 2025-05-28.

[7] Epoch AI. What skills does SWE-bench verified evaluate? https://epoch.ai/publications/what-skills-does-swe-bench-verified-evaluate, 2025. Accessed: 2025-06-01.

[8] Jatin Ganhotra. Cracking the code: How difficult are SWE-bench-verified tasks really? https://jatinganhotra.dev/blog/swe-agents/2025/04/15/swe-bench-verified-easy-medium-hard.html, 2025. Blog post, accessed: 2025-06-01.

[9] Shuyang Jiang, Yuhao Wang, and Yu Wang. SelfEvolve: A code evolution framework via large language models. *arXiv preprint arXiv:2306.02907*, 2023.

[10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. ICLR 2024.

[11] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Remi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[12] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[13] OpenAI. Introducing SWE-bench verified. https://openai.com/index/introducing-swe-bench-verified/, 2024. Accessed: 2025-06-01.

[14] OpenHands. SOTA on SWE-Bench verified with inference-time scaling and critic model. https://www.openhands.dev/blog/sota-on-swe-bench-verified-with-inference-time-scaling-and 2025. Accessed: 2026-05-28.

[15] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023.

[16] various. Dissecting the SWE-bench leaderboards: Profiling submitters and architectures of LLM- and agent-based repair systems. https://arxiv.org/abs/2506.17208, 2025. arXiv:2506.17208.

[17] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. *arXiv preprint arXiv:2402.01030*, 2024. ICML 2024.

[18] Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024. ICLR 2025.

[19] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

[20] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

[21] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. AGENTLESS: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[22] Hao Xu et al. Aligning LLMs for FL-free program repair. *arXiv preprint arXiv:2404.08877*, 2024.

[23] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[24] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023. ICLR 2023.

[26] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.